BÁO CÁO LAB 1

MÔI TRƯỜNG ESP32 & I/O CƠ BẢN

Thông tin sinh viên

- Họ và tên: Dương Quốc Thịnh
- MSSV: 23521498
- Nhóm: 14

---

1. Mục tiêu

- Tìm hiểu môi trường lập trình Arduino IDE.
- Thực hiện lấy mẫu tín hiệu cảm biến ở tần số cố định 100 Hz.
- Truyền dữ liệu thu thập được qua giao tiếp Serial.
- Sử dụng Python để nhận dữ liệu, lưu thành file CSV và vẽ đồ thị.
- Đánh giá độ ổn định của chu kỳ lấy mẫu thông qua thông số jitter.

---

2. Thiết bị sử dụng

Thiết bị	Mô tả
Vi điều khiển	ESP32 Dev kit V1
Cảm biến	Biến trở B10K
Phần mềm lập trình	Arduino IDE
Phần mềm xử lý dữ liệu	Python
Thư viện Python	pyserial, pandas, matplotlib

---

3. Nguyên lý thực hiện

Biến trở B10K được cấp nguồn và kết nối tới chân ADC của vi điều khiển.

Giá trị điện áp trên cần gạt biến trở thay đổi theo góc xoay:

- Điện áp nhỏ → ADC nhỏ.
- Điện áp lớn → ADC lớn.

Hệ thống sử dụng bộ Timer tạo ngắt định kỳ mỗi:

$$T_s = 10ms$$

tương ứng:

$$f_s = 100Hz$$

Mỗi lần ngắt xảy ra:

1. Đọc giá trị ADC.
2. Ghi nhận thời gian bằng hàm micros().
3. Gửi dữ liệu qua cổng Serial.

Dữ liệu gửi đi có dạng:

timestamp_us,sample,adc

Ví dụ:

10015,1,523

20019,2,526

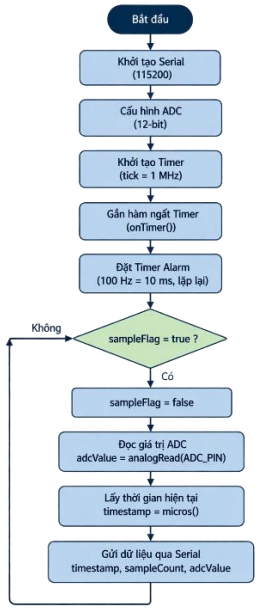
30021,3,529

trong đó:

- timestamp_us: thời gian lấy mẫu (µs)
- sample: số thứ tự mẫu
- adc: giá trị ADC

---

4. Lưu đồ thuật toán



5. Chương trình thực hiện

Chương trình ESP32

```
#define ADC_PIN 34

hw_timer_t *timer = NULL;
volatile bool sampleFlag = false;
volatile unsigned long sampleCount = 0;

void ARDUINO_ISR_ATTR onTimer()
{
  sampleFlag = true;
  sampleCount++;
}

void setup()
{
  Serial.begin(115200);

  analogReadResolution(12);

  // Timer tick = 1 us
  timer = timerBegin(1000000);

  timerAttachInterrupt(timer, &onTimer);

  // 100 Hz = 10 ms
  timerAlarm(timer, 10000, true, 0);
}

void loop()
{
  if(sampleFlag)
  {
    sampleFlag = false;

    int adcValue = analogRead(ADC_PIN);

    unsigned long timestamp = micros();

    Serial.print(timestamp);
    Serial.print(",");

    Serial.print(sampleCount);
    Serial.print(",");

    Serial.println(adcValue);
  }
```

```
}
```

Chương trình Python

Python thực hiện:

- Nhận dữ liệu Serial.
- Lưu dữ liệu thành file CSV.
- Tính chu kỳ lấy mẫu.
- Tính jitter.
- Vẽ đồ thị tín hiệu.

File read_serial.py

```
import serial
import pandas as pd
import matplotlib.pyplot as plt
from datetime import datetime

PORT = "COM6"
BAUD = 115200

ser = serial.Serial(PORT, BAUD)

data = []

print("Dang thu du lieu...")

try:
    while len(data) < 2000:      # lấy 20 giây
        line = ser.readline().decode().strip()

        try:
            t, n, adc = line.split(',')

            data.append([
                int(t),
                int(n),
                int(adc)
            ])

        except:
            pass

except KeyboardInterrupt:
    pass
```

```
ser.close()

df = pd.DataFrame(
    data,
    columns=["timestamp_us",
             "sample",
             "adc"]
)

filename = "dthinh.csv"

df.to_csv(filename, index=False)

print("Da luu:", filename)

plt.figure(figsize=(10,5))
plt.plot(df["timestamp_us"]/1e6,
         df["adc"])

plt.xlabel("Time (s)")
plt.ylabel("ADC")
plt.title("ADC Signal")
plt.grid(True)
plt.show()

period = df["timestamp_us"].diff()

period = period.dropna()

mean_period = period.mean()

jitter = period.std()

print()
print("===== KET QUA =====")
print(f"Chu ky trung binh = {mean_period:.2f} us")
print(f"Jitter (std) = {jitter:.2f} us")
```

File draw_adc_plot.py

```
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np

filename = "khiem.csv"

df = pd.read_csv(filename)
```

```
# Đổi thời gian về giây
time_s = df["timestamp_us"] / 1e6

# Tính chu kỳ thực tế
period = df["timestamp_us"].diff().dropna()

mean_period = period.mean()
jitter = period.std()

print("===== KET QUA =====")
print(f"Chu ky TB : {mean_period:.2f} us")
print(f"Jitter    : {jitter:.2f} us")

plt.figure(figsize=(12,6))
plt.plot(time_s, df["adc"], linewidth=1)

plt.title(
    f"ADC Signal\nMean Period={mean_period:.2f} us | Jitter={jitter:.2f} us"
)

plt.xlabel("Time (s)")
plt.ylabel("ADC Value")
plt.grid(True)

plt.tight_layout()
plt.savefig("khiem_plot.png", dpi=300)
plt.show()
```

File jitter.py

```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv("khiem.csv")

period = df["timestamp_us"].diff().dropna()

plt.figure(figsize=(10,5))
plt.hist(period, bins=30)

plt.xlabel("Sample Period (us)")
plt.ylabel("Count")
plt.title("Sampling Period Distribution")
plt.grid(True)

plt.savefig("khiem_jitter.png", dpi=300)
plt.show()
```

6. Kết quả thực nghiệm

6.1 Dữ liệu thu thập

Tần số lấy mẫu:

$$100Hz$$

Số mẫu thu thập:

$$2000\ mẫu$$

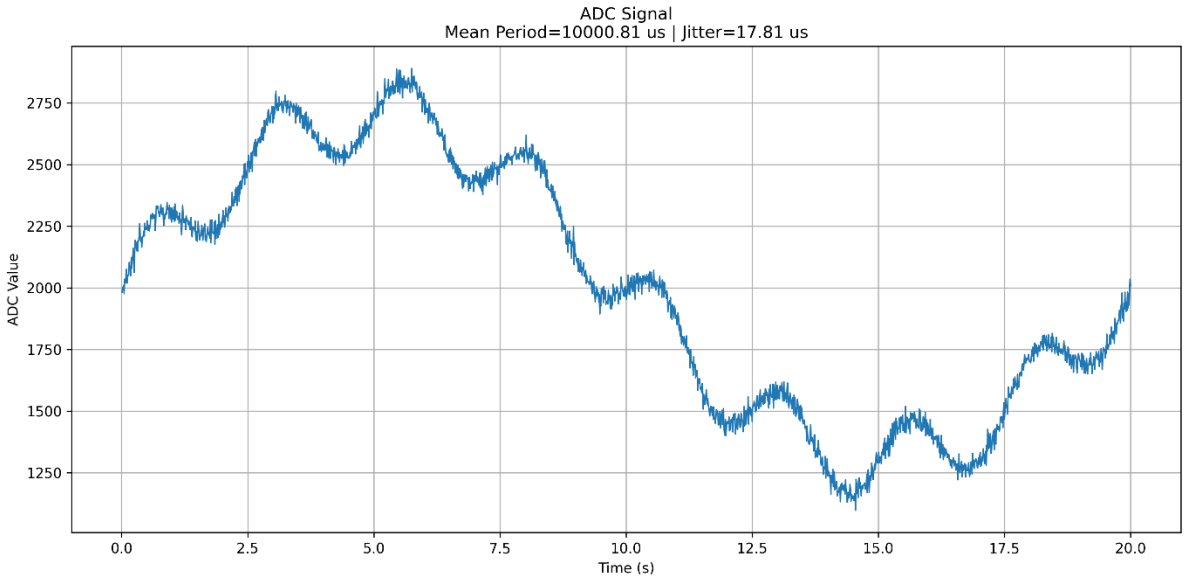
Thời gian đo:

$$20\ giây$$

Dữ liệu được lưu vào file:

dthinh.csv

6.2 Đồ thị tín hiệu ADC



Nhận xét:

Giá trị ADC thay đổi theo vị trí xoay của biến trở B10K.

Trong quá trình đo, khi xoay biến trở theo chiều tăng điện áp, giá trị ADC tăng tương ứng. Khi xoay ngược lại, giá trị ADC giảm.

Tín hiệu thu được tương đối ổn định và có nhiễu nhỏ.

6.3 Đánh giá chu kỳ lấy mẫu

Chu kỳ lấy mẫu được xác định từ hiệu giữa hai giá trị thời gian liên tiếp:

$$T_i = t_i - t_{i-1}$$

trong đó:

- t_i: thời điểm lấy mẫu thứ i (µs)
- T_i: chu kỳ lấy mẫu thực tế (µs)

Từ dữ liệu thu thập được trong file csv, chương trình Python tính toán chu kỳ lấy mẫu và jitter của hệ thống.

Bảng kết quả thống kê

Thông số	Giá trị
Chu kỳ lý thuyết	**10000 µs**
Chu kỳ trung bình	**10000.81 µs**
Jitter (độ lệch chuẩn)	**17.81 µs**
Chu kỳ nhỏ nhất	**9942 µs**
Chu kỳ lớn nhất	**10069 µs**

Nhận xét

Kết quả cho thấy chu kỳ lấy mẫu trung bình đạt 10000.81 µs, rất gần với giá trị lý thuyết 10000 µs tương ứng với tần số lấy mẫu 100 Hz. Sai số trung bình chỉ khoảng:

$$10000.81 - 10000 = 0.81\ \mu s$$

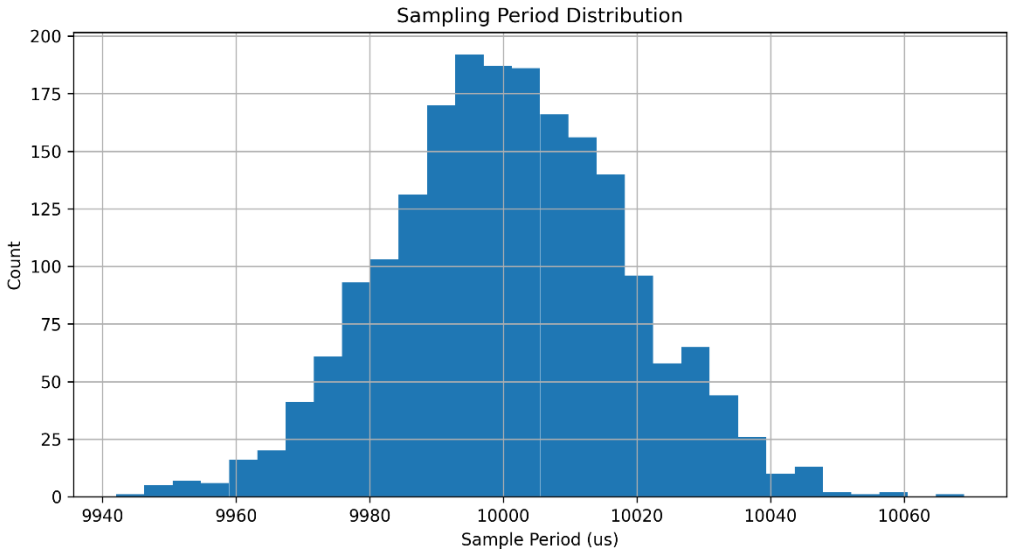
Jitter đo được là 17.81 µs, cho thấy độ dao động chu kỳ lấy mẫu là rất nhỏ so với chu kỳ danh định 10000 µs. Điều này chứng tỏ bộ định thời hoạt động ổn định và đáp ứng tốt yêu cầu lấy mẫu định kỳ của bài thí nghiệm.

Ngoài ra, chu kỳ lấy mẫu nhỏ nhất là 9942 µs và lớn nhất là 10069 µs, thể hiện vẫn tồn tại sai lệch nhỏ do thời gian xử lý chương trình và giao tiếp Serial, tuy nhiên mức sai lệch này không ảnh hưởng đáng kể đến quá trình thu thập dữ liệu.

Kết luận

Hệ thống đã thực hiện lấy mẫu với tần số xấp xỉ 100 Hz và duy trì độ ổn định cao trong suốt 2000 mẫu đo. Giá trị jitter nhỏ chứng tỏ phương pháp lấy mẫu bằng Timer ngắt phần cứng phù hợp cho các ứng dụng điều khiển và thu thập dữ liệu thời gian thực.

6.4 Đồ thị phân bố chu kỳ lấy mẫu



Nhận xét:

Các chu kỳ lấy mẫu tập trung quanh giá trị 10000 µs.

Độ lệch nhỏ cho thấy Timer hoạt động ổn định.

Biến động thời gian lấy mẫu chủ yếu do:

- Thời gian xử lý chương trình.
- Giao tiếp Serial.
- Sai số nội tại của bộ định thời.

---

7. Đánh giá và thảo luận

Ưu điểm:

- Hệ thống lấy mẫu đúng tần số yêu cầu 100 Hz.
- Dữ liệu được truyền và lưu ổn định.
- Python hỗ trợ trực quan hóa dữ liệu hiệu quả.

Hạn chế:

- Tín hiệu ADC vẫn xuất hiện nhiễu nhỏ.
- Jitter chưa bằng 0 do ảnh hưởng của hệ điều hành và thời gian thực thi chương trình.

---

8. Kết luận

Trong bài thí nghiệm này, đã thực hiện thành công việc thu thập dữ liệu từ biến trở B10K với tần số lấy mẫu 100 Hz.

Dữ liệu được truyền qua giao tiếp Serial, lưu thành file CSV và xử lý bằng Python để vẽ đồ thị và đánh giá chất lượng lấy mẫu.

Kết quả cho thấy chu kỳ lấy mẫu đạt gần giá trị lý thuyết 10 ms, jitter nhỏ, đáp ứng yêu cầu của bài thực hành.